University of California, Los Angeles
Anderson School of Management - MSBA

Business Analytics 402
Fall 2026

Prof. Ryan Rosario

Problem Set 1

Schemas and Single-Table Queries: Everything through `HAVING`

Query rules. All SQL is **PostgreSQL**. Unless an exercise says otherwise, every query is a `SELECT` and must not change the database. Use only what we have covered through Lecture 3: no joins, subqueries, `WITH` or temporary tables. Our rules for quizzes and exams apply:

1. Never use `SELECT *`. Name the columns you want.
2. In `ORDER BY` and `GROUP BY`, use column names or expressions, not numbers.
3. Do not use column aliases in `WHERE`, `GROUP BY` or `HAVING`.
4. In an aggregation, the grouping columns come first in the `SELECT`, then the aggregates. Every other column in the `SELECT` must be aggregated or in the `GROUP BY`.
5. Use `WHERE` for conditions on rows and `HAVING` for conditions on groups.

Sample output is given for most queries. Matching it is necessary, but not sufficient, for credit.

Part 1. Designing Schemas

Suppose you are working for the data team at *Bird Scooter*, a Santa Monica based startup that aims to revolutionize how people get around on wheels. How the Bird Scooter service works: a user installs an app on their phone and enters their credit card information. The app shows a map of deployed scooters nearby. The user scans a QR code on the scooter using the app, which activates the scooter for use and begins the clock for per-minute and per-use billing. The user rides the scooter for a distance, for a certain number of minutes. When the user is done with the scooter, they leave it somewhere, and mark the trip as complete in the app.

In this problem, we will create a SQL schema using the `CREATE TABLE` syntax. This means we also need to pick the proper data types for each column according to the parameters specified in lecture.

First, we need a table to represent a `scooter`. Each of the following statements is designed to give you a hint as to the proper data type.

1. Each scooter has an identifier `scooter_id`, a number. Since Bird is a startup, we assume that there are no more than 10,000 scooters.
2. Each scooter has a flag `status` that marks it as online, offline (broken etc.), and lost/stolen. Each scooter can have only one of these states at a time, and each scooter must have a state associated with it. When a new scooter is added to the service, it begins its life `offline` until it can be inspected for safety, and then deployed to a location.

We need a table to represent a `customer` (`user` is a system keyword so we will not use it):

1. Each customer has an identifier `user_id`, a number, and we assume that Bird has at most 500,000 users for now.
2. A user is just someone that installed the app, not necessarily someone that will use a scooter. Thus, they may, or may not have a credit card number `ccnum` (always 16 digits) and expiration date `expdate`. Expiration dates usually look like MM/YY, but to make this simpler so you can use a more apparent data type, it is safe to assume that the card expires at midnight (00:00) on the 1st of the month. (Note that we never store credit card numbers in plain text but we will do it here.)
3. Each user must have an `email` address. Assume an email address length is at most 100.

We need a table to represent a `trip`. To keep it simple, we will include start and end information in this table, but the end of trip information may be missing. Each trip is associated with:

1. a unique identifier `trip_id`, a number. Assume that the total number of rides is not small.
2. exactly one user `user_id` and exactly one scooter `scooter_id`.
3. a `start_time` and `end_time`, which includes the date.
4. a `pickup` and `dropoff` location. Note that a location is represented as a latitude and longitude, so you may need more than one column for each location.

Exercise 1.

(a) Write the SQL schema for the tables discussed above using `CREATE TABLE`. Specify a primary key, or composite primary key (if applicable) using the correct syntax. Specify the proper foreign key relationship on each table (if one exists) using the proper syntax. Try to minimize storage space because we can always promote later.

(b) Using these relations, draw the schema diagram. For an example of a schema diagram, see the Lecture 1 slides.

(c) Many apps only allow an email address to be used by exactly one user. Modify your schema to enforce this. **Hint:** look up constraints in the PostgreSQL documentation. Do you see anything new?

Exercise 2. Discussion.

(a) In the schema you just created, both the trip start and trip end were stored in the same table. Would it be a good idea to include the elapsed time in the `trip` table as well? Why or why not?

(b) Ultimately one use case for the data in the `trip` table is to charge the customer/user for the ride. Explain why it would be bad practice to include the charge in the `trip` table.

(c) List a few reasons why a Bird Scooter trip may not have an end time recorded.

(d) In this exercise we stored both the start time and the end time in the same table: `trip`. When the trip starts, a row is added to the `trip` table. Explain why we might want to divide this table into two: `trip_start` and `trip_end`. *Hint:* Pretend that you are the database system. What happens when a trip ends in both of these cases?

Next, suppose you are working for the data team at *Starship*, a San Francisco based startup that aims to revolutionize food delivery. How the Starship service works: a user installs an app on their phone and places an order at a participating restaurant within a geofenced region (e.g. UCLA campus). The user enters their credit card information, orders their meal and selects where they want the food delivered. A code is provided so that only the correct user can open the lid and retrieve their food. Due to limited supplies of robots, and high demand, the company wants to experiment with charging users based on the amount of time it takes to deliver the food from the restaurant to the user.

Exercise 3. We have the following relations for managing robots, users and orders:

```
Robot(robot_id, is_online)
# robot_id is some value that identifies a robot.
# is_online just indicates whether or not the robot is able to be used.

User(user_id, ccnum, expdate, email, name)
# user_id is some value that identifies the user
# ccnum is the user's credit card number (hashed, hopefully!) and may be null, and may change.
# expdate is the credit card expiration date and may also be null, and may change.
# email is the user's email address.
# name is the, well, user's name

Order(order_id, user, robot, restaurant, deliver_lat, deliver_lon, start_time, end_time)
# order_id is some value that identifies the order.
# user uniquely identifies the user that placed the order.
# robot uniquely identifies the robot that delivered the user's food.
# restaurant identifies where the user ordered food.
# deliver_lon is the longitude of the delivery location
# deliver_lat is the latitude of the delivery location
# start_time and end_time represent the start and end times of the delivery.
# we will ignore the details of the order for now.

Restaurant(restaurant_id, name, region)
# restaurant_id is some value that identifies the restaurant.
# name is the name of the restaurant
# region is the name of the geofenced region where the restaurant is located (e.g. UCLA)
```

Using these relations, draw the schema diagram. Clearly denote primary and foreign keys (note that primary keys were intentionally withheld in the specification above). *Hint:* There is a lot written above; however, remember that we are interested in **keys**.

Exercise 4. Now we will write the SQL schema for a **simplified** version of Starship, so be careful if you reference the diagram from the previous exercise. Each of the following statements is designed to give you a hint as to the proper data type.

First we need a table to represent a `robot`.

1. Each robot has an identifier `robot_id`, a number. Since Starship is a startup, we assume that there are no more than 30,000 robots.
2. Each robot has a flag `status` that marks it as online, offline, or lost/stolen. Each robot can have only one of these states at a time, and must have a state. The robot starts in the online state.

We need a table to represent an order. Each order is associated with:

1. a unique identifier `order_id`, a number. We expect there to be more than 50,000 orders.
2. exactly one user name (up to 255 characters) `user_name` and exactly one robot `robot_id`.
3. an email address containing at most 100 characters that must be present.

4. a credit card number `ccnum` (exactly 16 digits) and expiration date `expdate`. Expiration dates usually look like MM/YY, but to make this simpler you can use a more apparent data type. You do not need to worry about the time associated with the expiration date.

5. an `order_time` and `delivery_time`, which include the date. If the order is canceled after the robot is dispatched, or the robot fails, there will not be a delivery time.

6. a `delivery_location` as a GPS coordinate (a latitude/longitude pair). *Hint:* See the documentation on geometric types. Note that latitude and longitude together form a point on a Cartesian plane (actually a sphere, but we will assume a Cartesian plane for this problem). If the order is canceled or an error occurs with the robot, there may not be a delivery location stored.

To save time, we will not create a table for `restaurant` or `user`.

(a) Write the SQL schema for the tables discussed above using `CREATE TABLE`. Specify a primary key, or composite primary key using the correct syntax. Specify the proper foreign key relationship on each table (if they exist) using the proper syntax. Try to minimize storage space because we can always promote later.

(b) Explain the purpose of adding `ON UPDATE CASCADE` to the definition of a foreign key. What does it allow us to do in the referenced relation? What about `ON DELETE CASCADE`?

Part 2. Restaurants

For these exercises, you will use data from TripAdvisor about restaurants around the world (source). The table `ta_restaurant` contains data about each restaurant including its `name`, `city` and average `rating`.

Exercise 5. Write a query that extracts all restaurants that have `Lotus` in their name. You will want to look at the documentation for more info: pattern matching.

Exercise 6. Write a query that constructs the rating distribution – the number of restaurants that are in each rating bucket. Output the rating value as `rating` and the number of restaurants as `total`. The results should be arranged from lowest rating to highest rating.

```
 rating | total
--------+-------
     -1 |    41
      1 |   620
    1.5 |   384
      2 |  1437
    2.5 |  2717
      3 |  8514
    3.5 | 19661
      4 | 39707
    4.5 | 31277
      5 | 11225
        |  9582
(11 rows)
```

Exercise 7. Repeat the previous exercise, but only include ratings that are positive numbers, and do not include any NULLs.

```
 rating | total
--------+-------
      1 |   620
    1.5 |   384
      2 |  1437
    2.5 |  2717
      3 |  8514
    3.5 | 19661
      4 | 39707
    4.5 | 31277
      5 | 11225
(9 rows)
```

Exercise 8. Write a query that computes the average restaurant review score, and the number of restaurants, by city. The output should contain the columns `city`, `average_rating` and `num_rest` and should be sorted in descending order by average rating, breaking ties by city. Use only 3 digits after the decimal point. There should be 31 rows. Here are the first ten:

```
    city    | average_rating | num_rest
------------+----------------+----------
 Rome       |          4.232 |     5949
 Athens     |          4.208 |     1937
 Oporto     |          4.152 |     1580
 Krakow     |          4.129 |     1354
 Ljubljana  |          4.128 |      501
 Berlin     |          4.127 |     7078
 Amsterdam  |          4.118 |     3434
 Budapest   |          4.096 |     2606
 Vienna     |          4.068 |     3722
 Warsaw     |          4.068 |     2345
```

Part 3. Airbnb Listings

These exercises use the Inside Airbnb listings for Hawaii from Lecture 3, in the table `airbnb_listings` (36,452 rows). Each row is one listing. The columns you will need are:

`id`, `name`	the listing's identifier and title
`neighbourhood_group`	the **county**: Hawaii, Honolulu, Kauai or Maui
`neighbourhood`	the area within the county, e.g. Molokai or North Shore Kauai
`room_type`	Entire home/apt, Private room, Shared room or Hotel room
`price`	the nightly price in dollars (`NULL` when unknown)
`last_review`	the date of the most recent review (`NULL` if never reviewed)

Exercise 9. Molokai. Hawaii is a series of islands.

(a) Return the `name` of every listing on the island of Molokai, using `neighbourhood_group = 'Molokai'`. What happens?

(b) Look at the `neighbourhood_group` and `neighbourhood` values: return each pair once, sorted by `neighbourhood_group` and then `neighbourhood` (30 rows). Explain what you saw in part (a).

(c) Now return the names of the first 20 listings from Molokai.

Exercise 10. How many on Molokai? Count the listings on Molokai twice: once with `COUNT(DISTINCT name)` and once with `COUNT(id)`. Both return 199. Which one is guaranteed to be right, and why?

Exercise 11. From counties to islands. Make the output show the island rather than the county, without changing the table. Return each island once, as `island`, alphabetically.

(a) Use a **simple** `CASE` that turns Honolulu County into Oahu. (4 rows: Hawaii, Kauai, Maui, Oahu.)

(b) Molokai is its own island. Why can't the simple `CASE` from part (a) handle it? Write a **searched** `CASE` that does.

(c) What is still wrong? Fix it. (6 rows.)

Exercise 12. The whole state. Compute the average price (`average_price`, two digits after the decimal point) and the number of listings (`num_listings`) across all of the Hawaiian Islands, with a first column `region` whose value is `Hawaiian Islands`. If you get an error along the way, explain it.

```
      region      | average_price | num_listings
------------------+---------------+--------------
 Hawaiian Islands |        906.44 |        36452
```

Exercise 13. By island. Using your final `CASE` from Exercise 11, count the listings on each island and compute each island's average price (two digits after the decimal point). Sort by island.

```
 island  | num_listings | avg_price
---------+--------------+-----------
 Hawaii  |         8754 |    471.82
 Kauai   |         6270 |    627.90
 Lanai   |            9 |    271.75
 Maui    |        10986 |    971.56
 Molokai |          199 |    139.58
 Oahu    |        10234 |   1405.41
```

Exercise 14. Intuition check. Oahu averages over $1,400 a night. Does that seem right? Return the `name`, `neighbourhood`, `room_type` and `price` of the five most expensive listings, keeping the listings with no price out of the top five, and explain what you see.

Exercise 15. Counting.

(a) Count the listings in each `neighbourhood_group`, most listings first.

(b) Do the same for each `neighbourhood`, fewest listings first. (30 rows, from Lanai with 9 to Primary Urban Center with 7,263.)

```
 neighbourhood_group | num_listings
---------------------+--------------
 Maui                |        11194
 Honolulu            |        10234
 Hawaii              |         8754
 Kauai               |         6270
```

Exercise 16. County and room type. A classmate wants the number of listings and the average price for each `neighbourhood_group` and `room_type`, fewest listings first, and wrote:

```
SELECT
  neighbourhood_group,
  room_type,
  COUNT(id) AS num_listings,
  AVG(price) AS avg_price
FROM airbnb_listings
GROUP BY neighbourhood
ORDER BY num_listings;
```

(a) Why does it fail?

(b) Fix it, with the average price to two digits after the decimal point. There are 15 rows; here are the first five.

```
 neighbourhood_group |  room_type   | num_listings | avg_price
---------------------+--------------+--------------+-----------
 Maui                | Shared room  |            5 |     52.60
 Hawaii              | Shared room  |           14 |    130.93
 Honolulu            | Shared room  |           18 |    251.47
 Kauai               | Hotel room   |           24 |  20278.75
 Hawaii              | Hotel room   |           54 |  12256.77
```

Exercise 17. Oahu. For each neighbourhood on the island of Oahu, return the number of listings and the average price (two digits after the decimal point), sorted by neighbourhood.

```
    neighbourhood     | num_listings | avg_price
----------------------+--------------+-----------
 Central Oahu         |           46 |    199.63
 East Honolulu        |          184 |    463.72
 Ewa                  |         1007 |    576.98
 Koolauloa            |          587 |    464.92
 Koolaupoko           |          459 |    532.33
 North Shore Oahu     |          368 |    482.67
 Primary Urban Center |         7263 |   1751.69
 Waianae              |          320 |    337.48
```

Exercise 18. Kauai. Find all neighbourhoods on Kauai that average $300 or more per night and have more than 20 listings. Return the `neighbourhood`, number of listings and average price (two digits after the decimal point), from most expensive to least expensive.

```
   neighbourhood    | num_listings | avg_price
--------------------+--------------+-----------
 Kapaa-Wailua       |          929 |   1006.38
 Koloa-Poipu        |         1804 |    623.85
 Waimea-Kekaha      |           24 |    567.27
 North Shore Kauai  |         2646 |    530.70
 Lihue              |          867 |    514.67
```

Exercise 19. Two traps. Explain what goes wrong in each query.

(a)
```
SELECT
  neighbourhood_group,
  neighbourhood,
  COUNT(id) AS num_listings
FROM airbnb_listings
GROUP BY neighbourhood_group;
```

(b)
```
SELECT DISTINCT
  CASE neighbourhood_group WHEN 'Honolulu' THEN 'Oahu' ELSE neighbourhood_group END
FROM airbnb_listings;
```